16 · Solution Step 6: Training Recipe and Deployment

1. Goal of this step

Actually find good weights for the head, cheaply and reliably, and then run it without the teacher.

2. What the paper does

2a. Build the training data once

1. Run the LLM over the training conversations. At each retrieval trigger, let it generate the query and **save a trace**: the hidden states, the query text, the token count.
2. Encode each query text with the teacher to get **y**, and fetch the teacher's top-128 documents for rank distillation.
3. Store each trace as a file (11,017 in total), keyed by model name and settings.

This takes about **2 hours** on one GPU. After that the LLM doesn't even need to be loaded, and each training run of the head takes only **15–20 minutes**.

2b. Train the head

Setting	Value	Plain words
Optimizer	AdamW, $\beta = (0.9, 0.999)$, weight decay 10^{-4}	Gradient descent with per-weight step sizes and a gentle pull toward zero

Setting	Value	Plain words
Learning rate	$2 \times 10^{-4} \to 10^{-5}$, cosine schedule	Big steps first, tiny steps at the end
Epochs	80	80 passes over the training data
Batch size	16	16 queries per update
Gradient clipping	norm 1.0	Cap the size of any single update

(`math/12_updating_the_weights.md` works through gradient descent, Adam, clipping and the schedule with numbers.)

2c. Deploy

At search time: LLM writes the query → keep $\mathbf{H}$ → head → unit vector → search the unchanged index. The teacher is gone.

3. Why it is done this way

- **Caching** separates the slow part (running the 8B LLM) from the fast part (training a 25M head), so many experiments are cheap. That is how the paper affords 12 configurations.
- **Slow and long training wins.** The single biggest improvement in the paper came from the recipe, not from a loss:

Epochs / LR	R@10
30 / 3×10^{-4}	0.595
50 / 3×10^{-4}	0.552 (worse!)
80 / 2×10^{-4}	**0.607**

Epochs / LR	R@10
30 / 5×10^{-4}	0.001 (collapsed)

The paper reads this as: mapping one space into another involves delicate geometry that needs small, careful steps. Too big a learning rate destroys it, and training longer *without* lowering the rate overshoots.

4. Which problem it answers

Problems 1 and 2 **at inference time**: one large model, a 2 ms head, the same index.

5. What could go wrong

- **Sensitivity.** The gap between "best" (2×10^{-4}) and "collapsed" (5×10^{-4}) is only a factor of 2.5 in learning rate. On new data this might need re-tuning.
- **Retraining triggers.** Any change to the LLM (a new version, a fine-tune) or to the embedder means new traces and a new head.
- **Training still needs both models.** Only deployment is simplified.